Qiskit Certification — Pauli Operators, Circuits & Gate Library

30 Hard Multiple-Choice Questions

Study Group Prep Deck

August 29, 2026

Exam Practice — Question 1

Question 1

In Qiskit's symplectic representation $P = (-i)^{q+z \cdot x} Z^z X^x$, which single-qubit Pauli operator corresponds to $z = 1, x = 1$ (ignoring phase q)?

A. I

B. X

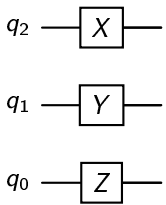
C. Z

D. Y

Exam Practice — Question 2

Question 2

For `Pauli('XYZ')`, which qubit does the left-most character `'X'` act on?



`Pauli('XYZ')` $= X \otimes Y \otimes Z$

A. Qubit 0 (the right-most position, least significant)

B. Qubit 1 (the middle position)

C. Qubit 2 (the left-most position, most significant)

D. It is ambiguous without also specifying the phase

Exam Practice — Question 3

Question 3

Given `Pauli((z, x, 2))`, i.e. phase index $q = 2$, what phase factor does this represent, using Qiskit's phase convention $q \in \{0, 1, 2, 3\} \to \{+1, -i, -1, +i\}$?

A. $+1$

B. $-i$

C. -1

D. $+i$

Exam Practice — Question 4

Question 4

According to the notebook, a `Pauli` object may only be composed with a `ScalarOp` (or multiplied) by which set of scalar values?

A. Any complex number with unit magnitude

B. Only $\{1, -1\}$ (real phases)

C. Only $\{1, -1, 1j, -1j\}$

D. Any of $\{1, -1, 1j, -1j, 1/\sqrt{2}, -1/\sqrt{2}\}$

Exam Practice — Question 5

Question 5

What happens to a `Pauli` operator's phase when `.adjoint()` is called on it, e.g. on `Pauli('iXYZ')`?

A. The phase is always discarded, leaving only the unsigned Pauli string

B. The Z/X bit pattern stays the same, but the phase is complex-conjugated (so $+i \to -i$)

C. Both the bit pattern and the phase are complex-conjugated

D. `.adjoint()` raises an error on Pauli operators with a non-trivial phase

Exam Practice — Question 6

Question 6

For `p1=Pauli('X')` and `p2=Pauli('iZ')`, what determines whether `p1.anticommutes(p2)` returns `True`?

A. Only the relative phase between the two operators

B. Whether the underlying X and Z Pauli strings anticommute, independent of the extra phase factor (and the number of qubits must match)

C. The alphabetical ordering of the Pauli labels

D. Whether both operators have been converted to matrix form first

Exam Practice — Question 7

Question 7

What is the default evolution picture used by `Pauli.evolve(clifford)` (i.e. without passing `frame='s'`)?

A. Schrödinger picture: $P' = C\,P\,C^\dagger$

B. Heisenberg picture: $P' = C^\dagger\,P\,C$

C. Both pictures are computed and averaged by default

D. There is no default; `frame` must always be specified explicitly

Exam Practice — Question 8

Question 8

What type of objects does `qc.data` contain, and what does each one hold?

A. Plain Python tuples of `(gate_name, qubit_index)`

B. `CircuitInstruction` objects, each containing the operation and the specific qubits/clbits it acts on

C. Raw matrix representations of each applied gate

D. String labels only, with no reference to which qubits are involved

Exam Practice — Question 9

Question 9

What is the key behavioral difference between `qc.append(other_gate, [0])` and `qc.compose(other_circuit, [0])` (with default arguments)?



A. They are functionally identical in every respect

B. append always modifies qc in place and returns an `InstructionSet`; compose (without `inplace=True`) returns a *new* circuit and leaves the original qc unmodified

C. compose always modifies qc in place, while append never does

D. append can only take `Gate` objects, never full `QuantumCircuits`

Exam Practice — Question 10

Question 10

Visually, how does a circuit built with `qc.append(sub_circuit.to_gate(), [...])` differ from one built with `qc.compose(sub_circuit, [...])`?

A. There is no visual difference; both always draw identically

B. `append` of a converted gate draws as a single opaque box (unless later decomposed); `compose` flattens the sub-circuit's gates inline into the drawing

C. `compose` always draws as an opaque box; `append` always draws expanded

D. Both require calling `.decompose()` before either will render at all

Exam Practice — Question 11

Question 11

What does `qc1.tensor(qc2)` accomplish, conceptually?

A. It multiplies the two circuits' unitary matrices together on the same qubits

B. It composes the two systems into a larger Hilbert space — equivalent to forming the tensor product of the two circuits' state spaces

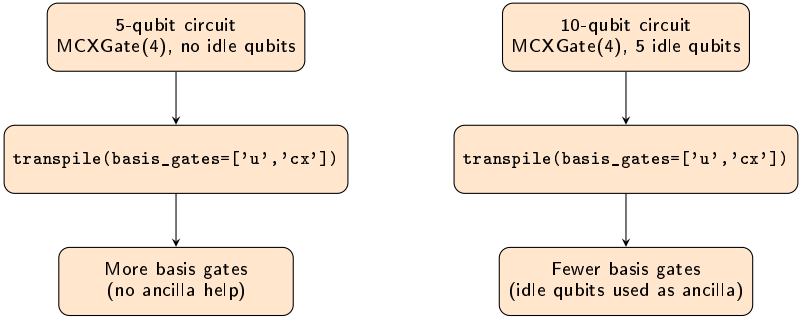
C. It appends qc2's gates directly on top of `qc1`'s existing qubits, in place

D. It is simply an alias for `qc1.compose(qc2)`

Exam Practice — Question 12

Question 12

An `MCXGate(4)` (5 total qubits touched) is appended into a circuit with no unused (idle) qubits, and separately into a circuit with 5 additional idle qubits available as potential ancillas. After `transpile(..., basis_gates=['u','cx'])`, what is the expected effect of the idle qubits on the transpiled gate count?



A. Idle qubits are always ignored by the transpiler and have no effect on gate count

B. The presence of extra idle qubits can let the transpiler use them as ancillas, generally reducing the

Exam Practice — Question 13

Question 13

Does inserting a `Barrier` instruction change the unitary computed by a quantum circuit?

A. Yes, a barrier always applies an identity-like but measurable phase shift

B. No — a barrier is purely a directive that visually separates parts of the circuit and blocks transpiler optimizations across it; it has no effect on the circuit's unitary

C. Yes, but only if the circuit is run on real hardware rather than a simulator

D. It depends on how many qubits the barrier spans

Exam Practice — Question 14

Question 14

What does the `measure` operation fundamentally do, and is it reversible?

A. It applies a unitary rotation to the qubit and is fully reversible

B. It measures the qubit in the computational basis, extracting the outcome and storing it onto a classical bit — this is a non-unitary, irreversible operation

C. It resets the qubit to $|0\rangle$ without recording any classical outcome

D. It is reversible only if immediately followed by a `reset`

Exam Practice — Question 15

Question 15

What is the key distinction between `qc.initialize(state, qubits)` and `qc.reset(qubit)`?

A. They are functionally identical — both simply set a qubit to $|0\rangle$

B. `reset` forces a qubit to $|0\rangle$ (non-unitary), while `initialize` can prepare an arbitrary (e.g. superposition) state and internally involves a reset followed by state preparation — also not a general unitary operation

C. `initialize` only works on classical bits, never qubits

D. `reset` can prepare arbitrary superposition states, while `initialize` only produces $|0\rangle$

Exam Practice — Question 16

Question 16

`DiagonalGate([1, 1j])` and `DiagonalGate([1, 1, 1, -1])` are constructed. What are their respective `num_qubits`?

A. 1 and 2

B. 2 and 4

C. 1 and 4

D. 2 and 2

Exam Practice — Question 17

Question 17

Inside `with qc.box():`, what does calling `qc.noop(8)` accomplish?

A. It applies an identity gate that changes the qubit's state

B. It marks qubit 8 as "used"/owned by that box block, even though no operation is applied to it

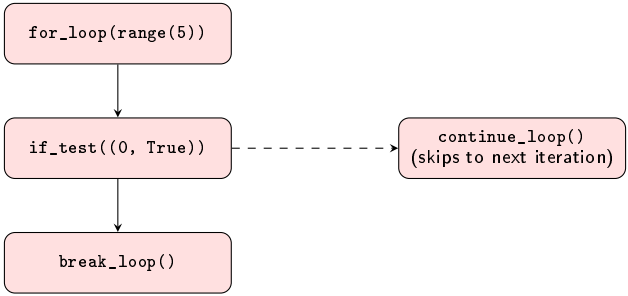
C. It removes qubit 8 from the circuit entirely

D. It is a synonym for `qc.barrier(8)`

Exam Practice — Question 18

Question 18

Inside `with qc.for_loop(range(5)) as i:`, what does calling `qc.break_loop()` inside a nested `with qc.if_test((0, True)):` block do?



A. It unconditionally terminates the entire circuit's execution
B. It exits the nearest enclosing loop early, conditioned on the classical value tested by `if_test`, enabling mid-circuit-measurement-driven dynamic control flow
C. It skips only the current loop iteration and continues with the next one

Exam Practice — Question 19

Question 19

What is the difference between `qc.measure_active()` and `qc.measure_all()`?

A. They behave identically in every case

B. `measure_active` measures only qubits that have been acted on (are "active"/non-idle) in the circuit, while `measure_all` adds measurements to *every* qubit, extending the classical register if necessary

C. `measure_all` only measures active qubits; `measure_active` measures every qubit

D. `measure_active` requires a classical register to already exist; `measure_all` does not

Exam Practice — Question 20

Question 20

What does `qc.remove_final_measurements()` do to a circuit?

A. It deletes every gate in the circuit, leaving only measurements

B. It strips trailing `measure` instructions (and associated unused classical resources) from the end of the circuit

C. It converts all measurements into `reset` operations instead

D. It has no effect unless the circuit was built with `measure_all()`

Exam Practice — Question 21

Question 21

For a circuit with 12 qubits and *no* classical bits, `qc.width()` returns 12. If the same circuit instead also had 3 classical bits added, what would `qc.width()` return?

A. 12, since `width()` only ever counts qubits

B. 15, since `width()` returns the total of qubits *plus* classical bits

C. 3, since `width()` only counts classical bits once any are present

D. `width()` would raise an error once classical bits are added

Exam Practice — Question 22

Question 22

What is required for `qc.estimate_duration(backend.target)` to return a meaningful value?

A. Nothing; it works on any circuit passed with any object

B. The circuit's gates must all be within the basis gates/timing information described by the supplied `target`, which supplies the per-gate durations used in the estimate

C. The circuit must first be converted into a `DAGCircuit`

D. The backend must be a genuine physical device — fake/generic backends are not supported

Exam Practice — Question 23

Question 23

After building `qc_scheduled = pm.run(qc)` using a preset pass manager *without* an explicit scheduling method, what values do `qc_scheduled.qubit_start_time(0)` and `qubit_stop_time(0)` return?

A. The exact wall-clock start/stop times measured on real hardware

B. Both remain at their default value of zero, because timing methods only return non-zero values after the circuit has been transpiled with an explicit `scheduling_method` such as `'alap'` or `'asap'`

C. A `RuntimeError` is raised immediately

D. They return `None` rather than any numeric value

Exam Practice — Question 24

Question 24

When `qc_h.control(1)` is called on a single-qubit `H`-gate circuit, how are the resulting two wires arranged?

A. The original qubit remains on wire 0 as the control; a new target wire is added at wire 1

B. The original qubit becomes the *target* and is moved to wire 1; the new control qubit occupies wire 0

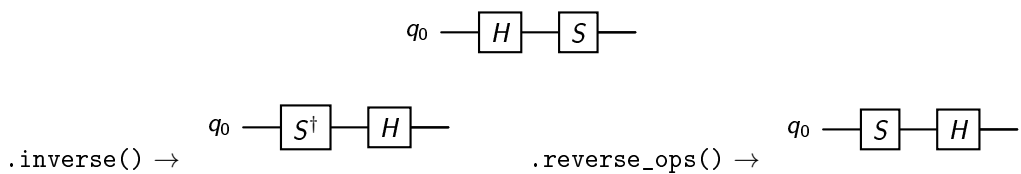
C. Both qubits become simultaneous controls with no defined target

D. `.control(1)` does not change wire indices; it only adds a label to the existing gate

Exam Practice — Question 25

Question 25

For a circuit `qc.h(0); qc.s(0)`, what is the key difference between the results of `qc.inverse()` and `qc.reverse_ops()`?



A. They produce exactly the same resulting circuit in every case

B. `inverse()` reverses the gate order *and* replaces each gate with its own inverse (giving `Sdg` then `H`); `reverse_ops()` only reverses the instruction order without inverting any gate (giving `S` then `H`)

C. `reverse_ops()` inverts each gate but keeps the original order; `inverse()` only reverses order

D. Both methods only work on circuits with no parameterized gates

Exam Practice — Question 26

Question 26

`TGate().power(2)` is appended to a 1-qubit circuit. What single standard gate is this mathematically equivalent to, and why?

A. An `X` gate, because T^2 flips the qubit

B. An `S` gate (phase gate with $\theta = \pi/2$), because $T = \mathrm{diag}(1, e^{i\pi/4})$ so $T^2 = \mathrm{diag}(1, e^{i\pi/2}) = S$

C. An identity gate, because any gate squared returns the identity

D. A `Z` gate, because $T^4 = Z$ implies T^2 is halfway to Z in an unrelated sense

Exam Practice — Question 27

Question 27

Given `qc.x(0); qc.z(2)` on a 3-qubit circuit, what specifically changes after calling `qc.reverse_bits()`, versus what would change after `qc.reverse_ops()` instead?

A. `reverse_bits()` changes the order gates are applied in; `reverse_ops()` changes which physical qubit each gate acts on

B. `reverse_bits()` relabels/reindexes the qubits (e.g. converting little-endian display to big-endian), leaving gate order and target logic unchanged, while `reverse_ops()` reverses the sequence of instructions but keeps each gate on its original qubit

C. Both methods are functionally identical for circuits with no multi-qubit gates

D. `reverse_bits()` only works on circuits containing exactly one gate

Exam Practice — Question 28

Question 28

What is the core structural difference between `TwoLocal` and `NLocal` circuits?

TwoLocal: rotation layer = 1-qubit gates on **ALL** qubits; entanglement layer = strictly 2-qubit gates

NLocal: blocks of *arbitrary* size $\leq$ num_qubits, repeated per the entanglement map

A. `TwoLocal` only supports 2 qubits total, while `NLocal` supports any number

B. In `TwoLocal`, the rotation layer consists of single-qubit gates applied to *all* qubits and the entanglement layer uses strictly 2-qubit gates; in `NLocal`, circuit blocks may have *arbitrary* size (up to the number of qubits), not restricted to 1- or 2-qubit blocks

C. `NLocal` is simply a deprecated alias for `TwoLocal`

D. `TwoLocal` uses `BlueprintCircuit` exclusively, while `NLocal` uses `QuantumCircuit` exclusively, with no other differences

Exam Practice — Question 29

Question 29

What is the key architectural difference that makes the ZZ feature map (ZZFM) generally harder to simulate classically than the plain Z feature map (ZFM)?

A. ZZFM uses more repetitions by default

B. ZZFM adds explicit two-qubit entangling gates (the $R_{ZZ}(\theta)$ rotation) between qubits, while ZFM consists only of alternating single-qubit Hadamard and phase-gate layers with no entanglement

C. ZZFM only works with an even number of qubits, while ZFM works with any number

D. ZFM only encodes real-valued features, while ZZFM encodes only complex-valued features

Exam Practice — Question 30

Question 30

In dense angle encoding (DAE), how many classical feature values can be encoded per qubit, and using which two rotation axes?

A. One feature per qubit, using only an R_Y rotation

B. Two features per qubit: one via an R_Y rotation (angle θ) and a second via an R_Z rotation (angle ϕ) applied to the same qubit

C. Three features per qubit, using R_X, R_Y, and R_Z together

D. Dense angle encoding does not use rotation gates at all; it uses only Hadamards